

Estación 2 — Manual del Estudiante

Descubrimiento y Definición del Problema

AI for Developers | 30X · Cohorte 4 · 27 de agosto, 2026

Manual del Estudiante

Cómo usar este documento

Este es el manual ampliado de la Estación 2. Es lo que viste y escuchaste en clase, profundizado para que puedas:

- **Repasar los frameworks** cuando estés trabajando solo durante la semana.
- **Ejecutar la práctica** de refinar tu PVB y co-crear tu PRD con AI, incluso si no terminaste durante la sesión.
- **Tener referencia rápida** para las próximas estaciones del programa.

El documento está organizado en seis partes y cinco anexos. La Parte III y IV son las más operativas — son los pasos concretos que debes ejecutar antes del viernes; la Parte IV (co-crear tu PRD con AI) es donde está el corazón de la sesión.

Si vas a leerlo de una sentada: **45 minutos aproximadamente**. Si vas a usarlo como referencia mientras trabajas en tu PVB y PRD: **lee la Parte III y IV con detalle, el resto lo consultás cuando lo necesites**.

PARTE I — POR QUÉ ESTÁS AQUÍ

1. La pregunta central

Estamos viviendo la ola tecnológica más grande desde internet. Más capital, más herramientas, más talento que nunca. Y aun así, la gran mayoría de los productos AI que se lanzan hoy van a estar muertos en 18 meses.

No es porque la tecnología no funcione. Funciona. No es porque no haya mercado. Lo hay.

Es algo más. Y la Estación 2 existe para que descubras exactamente qué es ese "algo más" — antes de invertir tres semanas construyendo algo que el mercado no recompensa.

La pregunta central de esta estación:

¿Qué separa a los que ganan de los que desaparecen?

Lo que sigue son los frameworks para que tu producto NO sea una de las historias que vamos a contarte ahora.

2. Tres caídas, una lección

Tres compañías. Tres errores completamente diferentes. El mismo resultado catastrófico. Las analizamos no para juzgarlas — la mayoría de equipos AI de 2025-2026 va a cometer al menos uno de estos errores. Las analizamos para que tú no.

Cómo leer estos casos. Lo que importa es la **historia y la lección** — qué pasó, por qué pasó, y qué te llevas para tu propio producto. Las cifras (valoraciones, recortes, revenue) van como **datos de apoyo**: dan textura, no son el eje. Si te quedás solo con un número, te perdiste el punto. Quedate con el patrón.

2.1 Chegg — El que no actuó

Chegg era EL recurso para estudiantes universitarios en Estados Unidos. Base de datos masiva de respuestas académicas. Millones de suscriptores pagando mensualmente. Modelo de negocio probado durante años. Los inversionistas los amaban.

Noviembre 2022: ChatGPT lanza. La reacción de Chegg fue esperar. "Vamos a ver qué pasa." "Es una moda." "Nuestro contenido curado es superior."

Nueve meses después, habían perdido el 90% de su valor de mercado.

Lo que pasa hoy (mayo 2026): la acción vale alrededor de un dólar. Venían de una valoración pico de 14 mil millones. Cortaron el 56% de su fuerza laboral. Intentaron un wrapper de GPT-4 llamado "CheggMate" — los estudiantes lo reconocieron en segundos como acceso pagado a tecnología gratis y no lo adoptaron. Ahora intentan pivotar a "Skilling", una plataforma de upskilling enterprise. Riesgo real de delisting del NYSE.

Por qué murió Chegg: los estudiantes descubrieron que podían hacerle la misma pregunta a ChatGPT y obtener una respuesta instantánea, personalizada, y gratuita. No había razón para seguir pagando. Chegg no murió porque la tecnología los superó. Murió porque no actuaron.

Primera lección:

En AI, la ventana de acción se mide en trimestres, no en años. "Esperar y ver" es una sentencia.

2.2 Jasper — El wrapper sin moat

Jasper sí actuó. Actuó rápido, levantó dinero rápido, creció rápido.

Jasper fue el poster child del boom de AI generativa. 125 millones de dólares levantados. Valuación de 1.2 mil millones. Eran "el futuro del AI writing." Las conferencias los invitaban como keynote. Los VCs peleaban por entrar en su ronda.

Pero había un problema fundamental que nadie quería ver: su moat era una interfaz bonita encima de la API de GPT. Cualquier desarrollador podía replicar eso en un fin de semana.

Y encima, su modelo de pricing era SaaS clásico: tarifa plana, 59 dólares al mes. Pero sus costos eran de inference — cada query costaba dinero real. Los mejores clientes eran los más caros.

Cuando ChatGPT se volvió mainstream: "¿Por qué pago \$59 al mes por un wrapper de algo que puedo usar gratis?"

Los números recientes confirman la caída: revenue cayó de \$120M a \$55M en un año. Pivotaron duro hacia enterprise — "Jasper Agents", "Jasper IQ" como capa de marca para empresas. Dicen que su ARR enterprise creció 4x post-pivot, pero desde una base mínima.

Segunda lección:

Si alguien puede replicar tu producto con la misma API en un fin de semana, no tienes moat. Y sin moat, no tienes producto — tienes un experimento temporal.

2.3 Duolingo — El que ejecutó mal

Duolingo sí tenía moat. Años de datos de aprendizaje de millones de estudiantes, patrones de errores por idioma, por región, por edad. Datos que ningún competidor podía comprar, replicar, o aproximar.

Tenían distribución: cientos de millones de usuarios activos. Tenían brand: la lechuza verde que te culpaba si no hacías tu lección. Todo alineado para ganar.

Entonces hicieron despidos de su equipo de soporte y los reemplazaron con tutores AI. La narrativa que le llegó al mercado fue devastadora: "Duolingo está reemplazando humanos con robots para ahorrar dinero."

No importó que la tecnología funcionara. No importó que los tutores AI fueran objetivamente buenos. Lo que importó fue la PERCEPCIÓN. La comunidad lo sintió como extractivo: "nos están quitando algo para quedarse con la diferencia."

El epílogo: 300,000 seguidores perdidos en semanas. En agosto de 2025, el CEO tuvo que publicar una aclaración diciendo que el memo interno "no dio suficiente contexto" y que no hubo despidos de empleados de planta. La empresa se silenció en redes durante semanas. Aun así, el daño ya estaba hecho.

Tercera lección:

Cómo implementas AI importa tanto como qué implementas. La tecnología estaba lista. La estrategia de comunicación no.

2.4 El patrón

Tres compañías. Tres errores completamente diferentes. El mismo resultado catastrófico.

Compañía	Tenía	Le faltó	Consecuencia
Chegg	Mercado dominante	Velocidad de acción	Casi delistado
Jasper	Capital + crecimiento	Moat real	De \$1.2B a sobrevivir
Duolingo	Data + distribución + brand	Ejecución de comunicación	Reputación dañada

La pregunta no es "¿deberíamos usar AI?" La pregunta es: "**¿Cómo construimos una ESTRATEGIA que sobreviva la velocidad brutal de esta ola?**"

Eso es lo que vas a aprender en lo que sigue de este manual.

3. Los dos caminos del builder AI

En este programa hay dos perfiles de builder. Ambos son igual de válidos. Antes de seguir con los frameworks, identifica cuál eres tú — porque los frameworks aplican a ambos, pero con lentes diferentes.

3.1 El Founder

Estás construyendo un producto para el mercado abierto. Trajiste tu propia idea o elegiste una del banco de productos. Tu objetivo es:

- Encontrar Product-Market Fit (PMF)
- Construir un moat defensible
- Eventualmente convertir esto en un negocio real

Tus referentes son los casos que vamos a estudiar: Perplexity, Clay, Cursor. Tu reto principal es encontrar un problema durable con un moat que se fortalezca con el tiempo.

3.2 El Forward Deployment Engineer (FDE)

Algunos de ustedes vienen financiados por una empresa. O quieren convertirse en el referente de AI dentro de su organización. Ese rol tiene nombre: **Forward Deployment Engineer**.

Lo inventó Palantir hace años. Hoy lo adoptan todas las empresas serias de AI. Es el ingeniero que entiende el problema del negocio profundamente Y sabe construir con AI. No hace un producto para el mercado abierto — construye soluciones internas que transforman operaciones.

Los números (2025):

- Postings de FDE crecieron **+800%** entre enero y septiembre de 2025
- Salesforce comprometió contratar **1,000 FDEs**
- Compensación promedio: **\$238,000** / staff-level: **\$630,000+**

Por qué hay tanta demanda: según un estudio de MIT Sloan publicado en 2025, **el 95% de los proyectos de GenAI en empresas NO llegan a producción**. Las empresas globales gastaron 684 mil millones de dólares en AI en 2025 — y 547 mil millones de eso no entregó valor. El costo promedio de un proyecto AI interno abandonado: 7.2 millones.

Y el dato más revelador: **el 77% de esas fallas son organizacionales, no técnicas**. La tecnología funciona. Lo que falla es entender el problema real, alinear stakeholders, y entregar algo que se adopte. El FDE existe para cerrar esa brecha. Y por eso es uno de los roles mejor pagados en tech ahora mismo.

3.3 Los frameworks aplican a ambos

Los frameworks que vienen aplican a ambos perfiles. La diferencia no es el framework — es quién es tu usuario y quién tiene el veto.

Usuario	El mercado	Un equipo o proceso interno
Quien paga	El cliente	El sponsor con presupuesto interno
Quien veta	El mercado (vía churn)	IT, compliance, el VP que controla el presupuesto
Éxito se mide en	Revenue, retención, moat	Adopción, impacto operativo, ROI demostrable
Failure mode	Nadie compra	Nadie adopta o IT lo bloquea

A partir de aquí, cada framework va a tener su "lens dual": cómo aplica al Founder y cómo aplica al FDE.

PARTE II — FRAMEWORKS DE EVALUACIÓN

Esta parte es el corazón teórico de la estación. Cinco frameworks que vas a aplicar a tu idea durante la semana — y vas a seguir usando durante todo el programa.

4. Test de durabilidad

¿Tu problema sobrevive a GPT-6?

Este es probablemente el test más importante. Si fallas acá, todo lo demás no importa.

Los productos que mueren tienen un patrón claro:

- "AI text summarizer"
- "AI grammar fixer"
- "AI meeting transcription"
- "AI image generator"

Son productos que resuelven **outputs**. Generan algo. Y ese algo se commoditiza rapidísimo porque el próximo modelo lo va a hacer mejor y gratis.

Los productos que sobreviven también tienen un patrón:

- Problemas de **workflow** (integración en un proceso)
- Problemas de **integración** específica
- Problemas de **contexto profundo** de industria

La regla en una línea:

Si tu problema es "generar X", se commoditiza.

Si tu problema es "integrar X en el workflow de Y", persiste.

Lens dual

Para el Founder:

Mira tu PVB. Mira cómo escribiste tu problema. ¿Suena a output o a workflow?

- ❌ "Generar reportes con AI" → zona de peligro, output

-  "Integrar análisis predictivo en el flujo de trabajo de analistas de riesgo crediticio en fintechs latinoamericanas" → durable, workflow

Para el FDE:

El mismo test aplica internamente. Si lo que quieres resolver es "resumir las minutas de reuniones" — eso lo va a hacer cualquier modelo gratis en seis meses. Tu solución va a ser obsoleta antes de que termine este programa.

Pero si es "integrar las decisiones de las reuniones de comité en el flujo de aprobación de presupuesto, con trazabilidad para auditoría" — eso requiere contexto profundo del negocio que ningún modelo genérico va a tener. Eso te vuelve irremplazable.

El FDE que identifica problemas de workflow se vuelve indispensable. El que automatiza outputs se vuelve redundante.

Cómo aplicarlo a tu idea

1. Escribe tu problema en una línea.
2. Pregúntate: "¿Esto va a estar resuelto en 6 meses por el próximo modelo de OpenAI / Anthropic / Google?"
3. Si la respuesta es "probablemente sí" → estás describiendo un output. Profundiza: ¿dónde se inserta este output en un workflow más grande? Ese workflow es tu producto real.
4. Si la respuesta es "no, porque requiere [contexto específico]" → estás en zona durable.

5. Las 5 preguntas con lens AI

Pasaste el test de durabilidad. Bien. Ahora hay que evaluar magnitud y oportunidad — porque no todos los problemas durables valen la pena en AI.

Cuando evalúas un pain point para un producto AI, necesitas un lens diferente al tradicional:

Pregunta 1 — Magnitud: ¿Cuántas personas sufren este problema?

No solo "cuántas personas en general". La pregunta para AI es: **¿aplica horizontalmente a través de industrias donde AI podría usarse?**

- **Founder:** ¿el dolor es lo suficientemente amplio para sostener un negocio?

- **FDE:** ¿cuántas personas en tu organización lo sufren? ¿Es un problema de un equipo de 5 o de 500?

Pregunta 2 — Frecuencia: ¿Qué tan seguido ocurre?

Clave para AI: si el problema no ocurre frecuentemente, no generas suficiente data para que tu modelo aprenda y mejore. Sin data recurrente, sin flywheel. Sin flywheel, sin moat.

- **FDE:** ¿el proceso ocurre diario, semanal, mensual? Los problemas diarios generan adopción más rápido y más data para mejorar.

Pregunta 3 — Severidad: ¿Qué tan malo es?

Las áreas donde AI brilla:

- Carga cognitiva pesada
- Reconocimiento de patrones
- Toma de decisiones repetitiva

Esos son los puntos donde AI realmente sobresale. Si tu problema involucra alguno de los tres, AI tiene ventaja estructural.

Pregunta 4 — Competencia: ¿Las soluciones actuales están limitadas por restricciones HUMANAS?

Esta es la pregunta de oro. ¿Velocidad? ¿Consistencia? ¿Escala? Si la respuesta es sí, AI puede trascender esas limitaciones de formas que el software tradicional nunca pudo.

- **FDE específicamente:** ¿el problema se resuelve hoy con un SaaS que alguien ya compró? ¿Con un Excel que alguien mantiene? ¿O no tiene solución porque nadie ha tenido tiempo? Las tres situaciones tienen estrategias muy diferentes.

Pregunta 5 — Contraste: ¿Hay quejas?

¿Los usuarios se quejan de falta de personalización, velocidad, o inteligencia? Esas quejas son señales puras de oportunidad AI.

- **FDE:** ¿qué dicen los usuarios internos cuando hablan del proceso actual? "Es muy lento." "No entiende mi caso." "Siempre tengo que ajustar manualmente." **Cada queja es un vector de oportunidad.**

Cómo aplicarlas

Toma 10 minutos. Recorre las 5 preguntas evaluando tu idea. Si alguna respuesta te incomoda, anótala. Esa es tarea de investigación para esta semana.

6. Las 3 trampas mortales

Estas son las trampas que matan startups AI y proyectos enterprise por igual. Sé honesto contigo mismo: ¿en cuál de estas tres trampas estás cayendo?

Trampa 1 — Red Ocean: la pelea que no puedes ganar

AI no está creando vastos mercados nuevos de la noche a la mañana. Está amplificando competencia en mercados existentes.

Para el Founder: si vas cabeza a cabeza contra un gigante, vas a perder.

- **Caso de muerte: Kite.** Fue pionero en AI code completion. Fue aplastado por GitHub Copilot — Microsoft tenía mejor data, mejor distribución, y la capacidad de subsidiar el producto cuando compitió.
- **Caso de éxito: CodiumAI.** En lugar de competir con Copilot en code generation, se enfocaron en el trabajo tedioso alrededor — tests y documentación. Encontraron el nicho COMPLEMENTARIO y levantaron 65 millones.

Para el FDE: tu Red Ocean es competir contra el equipo central de datos que ya tiene presupuesto, relación con el CTO, y track record. No compitas por su territorio. Complementa. Resuelve lo que ellos no tienen bandwidth de resolver. Sé el especialista, no el competidor.

Trampa 2 — Cool Demo: la ilusión de progreso

Generative AI hace peligrosamente fácil crear demos mágicas. Un par de llamadas API y tienes algo que parece el futuro. Pero la mayoría muere en el último 20% — el gap entre demo cool y producto confiable.

Para el Founder: Jasper vivió exactamente esto. El demo deslumbraba; el producto a escala se desangraba.

Para el FDE: esta es estadísticamente la trampa más mortal en enterprise.

El 95% de los pilotos de GenAI en empresas NO llegan a producción. (MIT Sloan, 2025)

Las empresas gastaron \$684B en AI en 2025; \$547B de eso no entregó valor. Costo promedio de un proyecto abandonado: \$7.2 millones.

¿El patrón? El VP queda deslumbrado con un POC. Aprueban presupuesto. Seis meses después, el proyecto está atascado porque nadie midió el impacto desde el inicio, nadie alineó a los usuarios reales, nadie pensó en compliance. El proyecto muere silenciosamente.

Tus jefes están aterrados de esto. Si llegas con el framework para evitarlo, eres oro puro.

Trampa 3 — Platform: construir sobre arena movediza

Si todo lo que construiste es una capa de UX sobre la API de alguien más y data pública, no estás construyendo un producto. Estás corriendo un experimento en tierra rentada. Una sola actualización de API puede replicar el 80% del valor de tu producto.

Para el Founder: wrappers de GPT con UI bonita murieron por docenas en 2024-2025.

Para el FDE: tu versión de la platform trap es depender de una herramienta que IT puede bloquear, o construir sobre un modelo que tu empresa decide no renovar la licencia. ¿Qué pasa si tu solución depende 100% de la API de Claude y la empresa decide cambiar a un proveedor diferente? ¿Qué tan portable es?

Auto-evaluación honesta

¿En cuál de estas tres trampas estás cayendo ahora mismo?

Si tu respuesta es "ninguna", relee con más cuidado. La mayoría cae en al menos una. Identificarla ahora es 10 veces más barato que descubrirla después de 3 semanas de construcción.

7. Los 3 moats que importan

Si te llevas una sola cosa de esta estación, que sea esto:

El mercado no recompensa features. Recompensa moats.

Features son temporales. Moats son permanentes.

Un moat es una ventaja competitiva que se fortalece con el tiempo y que los competidores no pueden copiar simplemente con dinero o acceso a la misma API.

Solo hay tres moats que importan en AI. Tres. Todo lo demás es ruido.

Moat 1 — Data Moat (el más durable)

Si tu producto genera datos únicos cada vez que alguien lo usa, estás construyendo una muralla que crece sola.

El flywheel:

```
graph LR; A[Más usuarios] --> B[Más data única]; B --> C[Modelos más inteligentes]; C --> D[Mejor experiencia]; D --> E[Mejor UX]; E --> A;
```

Más usuarios → Más data única → Modelos más inteligentes
↑
Mejor UX ← Mejor experiencia ←

Founder — Caso Spotify: no compite por su librería de 100 millones de canciones — eso es público. Compite por TU historial de escucha personal. Cada canción que skippeas, que guardas, que agregas a playlist. Esa data crea un Discover Weekly que se siente como magia. Ningún competidor puede replicarlo.

FDE — Tu data moat interno: ¿tu solución genera datos internos que mejoran con el uso? Cada interacción, cada corrección, cada decisión del equipo alimenta un modelo que se vuelve más preciso. Ese es tu data moat dentro de la organización. Y es lo que te hace irremplazable — no la herramienta, sino la inteligencia acumulada sobre cómo funciona tu empresa.

Moat 2 — Distribution Moat

En AI, la distribución es TODO. Si no eres dueño de ella, estás peleando por migajas contra ChatGPT y Claude.

Founder — Caso Notion: cuando agregaron AI, no necesitaron gastar millones en adquisición. Ya tenían 10 millones de usuarios embebidos en workflows. Solo "prendieron el switch" y la adopción fue instantánea.

FDE — Tu distribution moat: es estar embebido en el workflow diario del equipo. Si tu solución vive dentro de la herramienta que ya usan — Slack, el ERP, el CRM — la adopción es orgánica. Si requiere que abran otra herramienta o cambien su proceso, estás peleando contra la inercia. **Y la inercia gana casi siempre.**

Moat 3 — Trust Moat (el más subestimado)

Los usuarios no solo quieren AI poderoso; quieren AI predecible, seguro y confiable.

Founder — Caso Anthropic: se posicionó como "la empresa de AI safety" — eso les ganó clientes enterprise que no podían permitirse riesgo reputacional.

FDE — Tu trust moat: en tu organización, ¿quién controla el veto de confianza? ¿El compliance officer? ¿El CISO? ¿El regulador del sector? Si tu solución pasa ese filtro y la de un competidor no, tienes un moat que ningún feature puede superar. Identifica a esa persona temprano. Vuélvela aliada antes de tener producto.

Regla brutal

Solo puedes elegir UNO como primario.

No es que los otros dos no importen. Pero diluir tu foco en los tres significa no tener ninguno bien construido.

8. Casos de moats exitosos

8.1 Perplexity — Trust moat via citations

Perplexity no intentó "mejorar Google". Preguntaron desde cero: **¿cómo debería verse la búsqueda en un mundo AI-first?**

Su insight fue brillante:

- LLMs pueden responder preguntas directamente, pero alucinan.
- Search tradicional da links verificables pero requiere trabajo del usuario.
- ¿Y si combinas lo mejor de ambos?

Crearon un answer engine que da respuestas directas pero cada afirmación tiene una cita verificable. Eso creó un trust moat orgánico.

Y su distribución es viral: los outputs son tan útiles y bien formateados que la gente los comparte en redes sociales. Cada screenshot compartido es marketing gratuito.

Los números (mayo 2026):

- **\$21B** valoración
- **\$200M** ARR
- **780M** queries / mes

Pero: los está demandando el New York Times, BBC y Dow Jones por scraping. El moat de trust que construyeron ahora está siendo testeado por riesgo legal.

Lección: incluso los moats tienen vulnerabilidades. Identifícalas antes de que el mercado lo haga por ti.

8.2 Clay — Category creation moat

Clay no construyó una herramienta de CRM enrichment. **Inventaron un rol profesional:** el "GTM Engineer" (Go-To-Market Engineer). Escribieron el handbook. Definieron las competencias. Crearon la identidad del operador.

¿Por qué funciona? Porque cuando defines el rol, defines el toolset. Cada startup que contrata un GTM Engineer ahora pone Clay como el sistema operativo obvio para ese puesto.

Los números (mayo 2026):

- **\$3.1B** valoración (Serie C, agosto 2025)
- **\$100M** ARR en solo 2 años
- **Clientes:** OpenAI, Anthropic, Cursor — empresas con recursos para construir sus propias soluciones, pero compran Clay porque es el sistema operativo de un rol que Clay mismo definió

Pregunta para ti: ¿pueden ustedes definir un nuevo rol o categoría? ¿Qué identidad profesional podría crear su producto?

Para los FDEs: si pueden posicionarse como el "AI implementation lead" o el "applied AI engineer" dentro de su organización — y construir el playbook de cómo se hace AI en su empresa — están haciendo category creation a escala interna. Eso es exactamente lo que los hace irremplazables.

9. Elegir tu moat

Toma la decisión ahora. Solo uno. Las preguntas guía:

Moat	Founder pregunta	FDE pregunta
Data	¿Qué data única genero que competidores no pueden comprar?	¿Mi solución genera datos que mejoran con el uso del equipo?
Distribution	¿Dónde estoy embebido que es difícil de sacar?	¿Vivo dentro del workflow diario o requiero que cambien de herramienta?
Trust	¿Qué reliability ofrezco que otros no?	¿Paso el filtro del compliance officer y el CISO?

Si no puedes elegir, esa es la pregunta más importante de esta semana. Más importante que cualquier feature, cualquier diseño, cualquier línea de código.

10. Las 5 preguntas incómodas

Estas preguntas son incómodas A PROPÓSITO. Si no puedes responderlas, tienes tarea de investigación.

Mejor descubrir debilidades ahora que después de tres semanas de construcción.

Pregunta 1 — ¿El problema desaparece en 12 meses?

Si tu producto es sobre outputs, la respuesta probablemente es sí. Si es sobre workflows, probablemente sobrevive.

Red flag: si tu respuesta es "bueno, pivotearemos", no tienes una estrategia. Tienes una opción libre que no has ejercido.

Pregunta 2 — ¿Quién es dueño de la data?

- **Founder:** ¿viene de usuarios (propio)? ¿De terceros (riesgoso)? ¿De scraping (muy riesgoso)? ¿Qué pasa si tu fuente de data te corta acceso mañana?
- **FDE:** ¿los datos que necesitas son accesibles? ¿Quién controla el acceso? ¿IT, el dueño del sistema, un proveedor externo? Si dependes de datos que no controlas, esa es una dependencia que tienes que manejar desde el inicio.

Pregunta 3 — Si fallas públicamente, ¿quién se avergüenza?

Esto revela quién controla el veto de confianza.

- **Founder:** en healthcare es el regulador. En fintech, el compliance officer. En enterprise, el CISO. Si no sabes quién veta, no conoces tu mercado.
- **FDE:** ¿quién de tu organización queda expuesto si tu solución falla? Ese es tu stakeholder más importante. Y deberías tenerlo de aliado **desde el día uno** — no después del lanzamiento.

Pregunta 4 — ¿Un competidor puede replicar lo que haces en 6 semanas?

Si tu producto es un wrapper de GPT con UI bonita, la respuesta es sí.

FDE específicamente: ¿qué tienes que un proveedor SaaS externo no puede ofrecer? La respuesta casi siempre es: **contexto del negocio**. Esa es tu ventaja. Si no la estás explotando, cualquier SaaS te reemplaza.

Pregunta 5 — ¿Cómo se rompe trust a escala?

Si no puedes articular tu failure mode más probable a 100 veces usuarios — o a 100 veces el volumen actual — no estás listo para escalar.

La regla del manual

Un campo en blanco honesto vale más que una respuesta inventada.

Si no tienes la respuesta hoy, déjala en blanco y anota la pregunta que necesitas investigar. Eso es información, no fracaso.

PARTE III — REFINAR TU PVB (paso a paso)

Esta es la primera parte operativa del manual. **Es la práctica más importante de la semana.** Si solo haces una cosa de este documento, haz esta.

11. ¿Qué es un Product Vision Board?

Un Product Vision Board (PVB) es un canvas de una página que captura los elementos fundamentales de la estrategia de tu producto **antes** de empezar a construirlo.

Es la versión condensada de tu pensamiento estratégico. Sirve para tres cosas:

1. **Forzarte a pensar antes de construir.** Si no puedes llenar el canvas con claridad, no estás listo para escribir código.
2. **Crear alineación.** Si trabajas con equipo o stakeholders, todos miran el mismo documento.
3. **Tener una referencia que envejece mal.** Cuando una asunción crítica cambia, vas a tener que actualizar el PVB. Eso es señal de que tu estrategia está evolucionando, no de que está mal.

El PVB no reemplaza un PRD. El PRD viene después, con el detalle de features. El PVB es el "qué problema resolvemos para quién, cómo, y por qué nos defendemos."

12. Por qué refinar tu PVB después de esta sesión

Llegaste a la Estación 2 con un PVB que armaste antes. Es probable que ese PVB tenga al menos algunos de estos problemas:

- El problema descrito es un **output**, no un workflow
- No identifica al **veto holder**
- No elige un **solo moat primario**
- No tiene una **arena competitiva** clara (Pioneer / Disruptor / Enhancer)
- El **modelo económico** asume que los márgenes de SaaS aplican

Todo eso es normal. Es por eso que existe esta estación.

El refinamiento no es "hacerlo bonito". Es **estresar cada campo con los frameworks que acabas de aprender** y aceptar honestamente que algunos campos quedan en blanco hasta que investigues más.

13. Refinamiento campo por campo

Toma 15-25 minutos. Abre tu PVB. Recorre estos pasos en orden.

13.1 Campo PROBLEMA

Lo que escribiste antes: probablemente algo como "los usuarios necesitan X" o "queremos usar AI para Y".

Lo que debería decir: un dolor específico que sobrevive al próximo modelo.

Pasos:

1. Aplica el test de durabilidad: ¿es output o workflow?
2. Si es output, profundiza: ¿en qué workflow más grande se inserta?
3. Reescribe el problema en una línea, **incluyendo la industria/contexto específico**.
4. Asigna un **Durability Score** del 1 al 5 (5 = sobrevive 3+ generaciones de modelos foundation).
5. Si tu score es menos de 3, el problema no está listo. Anota qué necesitas investigar.

Antes: "Los analistas tardan mucho en hacer reportes." **Después:** "Los analistas de riesgo crediticio en fintechs LatAm pasan 60% de su semana reconciliando data entre 4 sistemas para producir el reporte de cartera mensual." (Durability: 4)

13.2 Campo SEGMENTO

Lo que escribiste antes: probablemente "empresas" o "PYMES" o "usuarios B2B".

Lo que debería decir: un segmento tan específico que podrías nombrar 5 empresas representativas.

Pasos:

1. Pregúntate: ¿qué tienen en común mis 10 usuarios ideales? Tamaño, industria, geografía, rol exacto, herramientas que ya usan, presupuesto.
2. Escribe el segmento como una frase larga: "Equipos de [rol] en [tipo de empresa] de [tamaño] en [geografía] que usan [herramientas] y pagan [rango de presupuesto]".
3. Verifica nombrando 5 empresas o equipos reales que encajan. Si no puedes, tu segmento sigue siendo demasiado vago.

Antes: "Equipos de legal en empresas grandes." **Después:** "Equipos de legal in-house en startups Series A-B (US/LatAm), 2-5 abogados, que usan Notion + Google Docs, con presupuesto para tools entre \$5K-\$50K anuales."

13.3 Campo VETO HOLDER

Lo que escribiste antes: probablemente nada. Este campo no existía en versiones tempranas del PVB.

Lo que debería decir: la persona o función que puede matar la adopción aunque el end-user ame el producto.

Pasos:

1. Pregúntate: si mi end-user ama el producto pero alguien dice "no", ¿quién es ese alguien?
2. En distintos contextos: - Healthcare: regulador, jefe de compliance - Fintech: compliance officer, CFO - Enterprise: CISO, IT, procurement - Internal FDE: VP del área, IT, CFO
3. Identifica también qué necesita esa persona para decir "sí". Eso es trabajo de investigación.

13.4 Campo MOAT PRIMARIO

Lo que escribiste antes: probablemente "mejor UX" o "nuestra tecnología".

Lo que debería decir: uno de los tres — Data, Distribution, o Trust.

Pasos:

1. Lee de nuevo la Parte II, sección 7 de este manual.
2. Elige UNO. No tres. UNO.
3. Si no puedes elegir, esa es tu investigación más importante de la semana.
4. Escribe una frase corta justificando: "Nuestro moat es **[Data/Distribution/Trust]** porque [razón concreta]."

Ejemplo: "Nuestro moat es **Data** porque cada análisis de cartera que procesamos genera labels propietarios de tipos de riesgo en fintechs LatAm que ningún proveedor externo tiene."

13.5 Campo ARENA COMPETITIVA

Lo que escribiste antes: probablemente nada o "competimos contra X".

Lo que debería decir: uno de los tres — Pioneer, Disruptor, o Enhancer.

Arena	Definición	Cuándo elegirla
Pioneer (AI-Native)	Creas un mercado nuevo que no podría existir antes de AI	Tu reto principal es creación de categoría
Disruptor (AI-Disrupted)	Usas AI para reimaginar un workflow existente 10x mejor	Hay mercado pero está mal servido
Enhancer (AI-Enhanced)	Eres un incumbent agregando AI a tu producto	Probablemente NO eres tú si recién empiezas

Para startups en este programa: usualmente Disruptor. A veces Pioneer. Casi nunca Enhancer (eso requiere ya tener producto).

13.6 Campo UX PARADIGM

Lo que escribiste antes: probablemente nada.

Las opciones:

- **Assistant** — el usuario está en control, AI sugiere (ej: GitHub Copilot)
- **Agent** — AI ejecuta tareas autónomamente dentro de límites (ej: AI SDR que envía emails)
- **Autonomous** — AI corre sin supervisión (ej: fraud detection que auto-bloquea)
- **Embedded** — AI mejora el producto sin que el usuario sepa que es AI (ej: Netflix recommendations)

Elige uno. Si dudas entre dos, anota cuál es el camino "fácil" para empezar y cuál es el "ideal" final.

13.7 Campo MODELO ECONÓMICO

Lo que escribiste antes: probablemente nada o "vendemos suscripciones".

Lo que debería decir:

- **Founder:** modelo de pricing concreto. Estima costo por usuario al mes. Estima revenue por usuario al mes. Calcula gross margin.
- **FDE:** "pricing" no aplica directamente. Tu equivalente es ROI demostrable. ¿Cuánto le ahorra a la organización? ¿En tiempo? ¿En errores? ¿En headcount que se redirige a trabajo de mayor valor?

Regla brutal: si tus AI costs > 30% del revenue por usuario (o si tu ROI no es 5x+ para el FDE), estás en danger zone.

13.8 Campo 3 RIESGOS CRÍTICOS

Lo que escribiste antes: probablemente nada o "competencia, recursos, timing".

Lo que debería decir: tres riesgos específicos derivados de las 5 preguntas incómodas (sección 10).

Ejemplos de buenos riesgos críticos:

- "Si OpenAI lanza un Operator-style agent específico para fintech, perdemos 70% de nuestro diferenciador en 6 meses."
- "Nuestro pipeline de data depende de una integración con Bloomberg que podríamos perder si cambian su política de API."
- "El compliance officer del cliente target necesita certificación SOC 2 Type II que nos toma 6 meses obtener — no podemos vender hasta entonces."

14. Checklist final de refinamiento

Antes de pasar al PRD, valida tu PVB contra esta checklist:

- ☐ **PROBLEMA:** Durability Score ≥ 3 , escrito en una línea, contexto específico de industria
- ☐ **SEGMENTO:** Puedes nombrar 5 empresas/equipos reales que encajan
- ☐ **VETO HOLDER:** Identificado por rol Y por qué necesita para decir "sí"
- ☐ **MOAT PRIMARIO:** Solo UNO elegido, con justificación de una línea
- ☐ **ARENA:** Pioneer / Disruptor / Enhancer
- ☐ **UX PARADIGM:** Assistant / Agent / Autonomous / Embedded
- ☐ **MODELO ECONÓMICO:** Pricing concreto (Founder) o ROI demostrable (FDE)
- ☐ **3 RIESGOS CRÍTICOS:** Específicos, derivados de las 5 preguntas incómodas

Si algún campo quedó en blanco, anota la **pregunta de investigación** debajo. Esa es tu tarea para esta semana.

Un campo en blanco honesto vale más que una respuesta inventada.

PARTE IV — CREAR TU PRD (paso a paso)

15. ¿Qué es un PRD?

PRD = Product Requirements Document.

Es el documento que traduce tu visión estratégica (PVB) en **una especificación completa del producto** — qué es, para quién, cómo se diferencia, cómo se mide — con foco explícito en qué entra en el MVP que cumple el objetivo.

Importante: el PRD describe el **producto completo**, no solo lo que vas a shippear en las próximas semanas. Dentro de esa descripción completa, hay una sección dedicada al **MVP Scope** que delimita qué se construye primero. El resto queda como roadmap futuro, pero documentado.

15.1 Definición operativa

Un PRD bien escrito responde estas preguntas sobre el producto completo:

1. **¿Qué problema resolvemos y para quién?** (visión + ICP)
2. **¿Cómo nos diferenciamos?** (propuesta de valor única + competencia)
3. **¿Qué construye el producto completo?** (módulos, features, casos de uso, user journeys)
4. **¿Qué entra en el MVP?** (subset priorizado con MoSCoW: Must / Should / Could / Won't)
5. **¿Cómo sabemos si funcionó?** (métricas de éxito, plan de evaluación, riesgos)

15.2 Historia y propósito

El PRD nació en empresas de software tradicional (Microsoft, IBM en los 90s) para alinear PMs, ingenieros, diseñadores y QA antes de empezar a construir. La idea era: si todos firmamos el mismo documento, no hay sorpresas a mitad del proyecto.

En empresas modernas (Stripe, Notion, Linear) el PRD se redujo a un documento más conciso que vive en Notion o Linear, pero **sigue describiendo el producto completo**. Lo que cambió fue la disciplina de no inflarlo con detalle innecesario.

En 2025-2026 con AI, el PRD es más importante que nunca — pero por una razón diferente. Ya no es solo para alinear humanos. Es el **contexto que le das a la AI** cuando estás co-construyendo. Mientras más completo y específico sea tu PRD, mejor código y mejores artefactos vas a producir con tu agente.

15.3 Qué incluye un buen PRD

El template oficial del programa estructura el PRD en **13 segmentos** que se generan secuencialmente con AI. Lo verás en detalle en la sección 17.

Tipo de sección	Propósito
Visión y problema	Qué resuelve el producto completo, para quién, por qué ahora
Diferenciación	UVP, competencia, posicionamiento
Casos de uso y journeys	Cómo se usa el producto completo
Principios no negociables	Reglas que el producto no puede violar
MVP Scope (MoSCoW)	Subset del producto completo que entra en v1
Especificación funcional	Módulos, features, arquitectura funcional
Métricas y evaluación	Cómo medimos éxito y calidad del agente
Riesgos y plan 30/60/90	Qué puede romperse y cómo entregamos

15.4 Qué NO incluye un PRD

- **Diseño visual detallado** — eso va en Figma o en docs de diseño aparte
- **Arquitectura técnica completa** — eso es el output del Prompt 2 (Arquitectura). En C4 el diseño técnico se trabaja en la Estación 5, *Diseñando el Cómo*
- **Backlog de ingeniería con stories detalladas** — eso es el output del Prompt 3 (Backlog)
- **Plan de marketing** — otro documento

16. ¿Por qué un PRD en AI es diferente?

Un PRD tradicional asume comportamiento determinístico: si input X, output Y, siempre. AI no funciona así.

Cuatro diferencias clave:

16.1 Las features pueden tener comportamiento probabilístico

Un botón en software tradicional siempre hace lo mismo. Una llamada a un LLM puede devolver respuestas distintas la misma pregunta. Tu PRD tiene que aceptar esto y especificar **rangos aceptables** en vez de outputs exactos.

Ejemplo:

- ❌ "El AI debe extraer el número de cliente."
- ✅ "El AI extrae el número de cliente con precisión $\geq 95\%$ en muestra de 500 documentos. Si la confianza es $< 80\%$, escalar a revisión humana."

16.2 Hay que definir failure modes explícitamente

¿Qué pasa cuando la AI alucina? ¿Cuándo dice algo ofensivo? ¿Cuándo es muy lenta? ¿Cuándo es muy cara?

Tu PRD tiene que tener una sección dedicada a esto, no solo "happy path".

16.3 Hay que pensar en costos por uso, no solo desarrollo

En SaaS, el costo marginal por usuario es $\sim \$0$. En AI, cada query cuesta dinero real. Tu PRD debe estimar:

- **Costo por interacción** (tokens $\times$ precio)
- **Costo por usuario activo / mes** (interacciones promedio $\times$ costo)
- **Margen objetivo** (revenue $-$ costo)

16.4 Hay que diseñar para iteración rápida

En AI los modelos cambian cada 6 meses. Lo que es state-of-the-art hoy será baseline en seis meses. Tu PRD debe estar diseñado para que cambiar de modelo (o de framework) no requiera reescribir el producto entero.

17. Estructura del PRD: los 13 segmentos

El template oficial del programa estructura el PRD en **13 segmentos** que la AI genera secuencialmente, uno por uno, esperando tu aprobación en cada paso. Estos segmentos cubren el producto completo, con el **MVP Scope (Segmento 8)** como subset priorizado.

#	Segmento	Qué contiene
1	One-Liner + JTBD	Frase de producto + Job to be Done + misión
2	Contexto y Problema	Dolores con datos, "¿por qué ahora?", alternativas actuales
3	ICP Detallado	Firmographics, buyer personas, pains, triggers, objeciones
4	UVP y Diferenciadores	Propuesta de valor única, brecha de mercado, posicionamiento
5	Casos de Uso Top 5	Actor, trigger, steps, resultado, KPI impactado
6	Principios No Negociables	Reglas de diseño que el producto no puede violar
7	User Journeys	Happy paths + edge cases (interrupción, escalación humana)
8	MVP Scope (MoSCoW)	Must / Should / Could / Won't — el subset que se construye primero
9	Especificación Funcional	Módulos, features, roles, diagrama funcional
10	Métricas de Éxito	North Star + KPIs de activación/retención/calidad
11	Plan de Evaluación del Agente	Dataset, criterios de calidad, QA, red-teaming
12	Riesgos y Mitigaciones	10 riesgos top con probabilidad, impacto, plan
13	Plan de Entrega 30/60/90 Días	Qué se construye, valida, entrega y mide

Punto crítico sobre el Segmento 8 (MVP Scope): Los Segmentos 1-7 y 9-13 describen el producto completo. El Segmento 8 es donde delimitas explícitamente qué subset entra en v1. El resto del PRD sigue existiendo como roadmap futuro documentado — no se borra. El MVP se prioriza con **MoSCoW** (Must / Should / Could / Won't) — es la forma de decidir qué entra a v1 sin matar el roadmap.

Tres precisiones que evitan errores comunes en los segmentos:

- **Job to be Done (Segmento 1):** la gente no compra el producto, compra el "trabajo" que el producto hace por ellos. El clásico: nadie quiere un taladro, quieren el hueco en la pared — y en realidad ni eso: quieren el cuadro colgado. Otro: nadie quiere un martillo, quiere meter las puntillas y, al final, colgar el cuadro. O Microsoft Word: nadie quiere Word, quiere escribirle la carta al colegio del hijo. Definí tu producto por el trabajo que resuelve, no por sus features.
- **User Journeys (Segmento 7):** describen cómo una persona real recorre tu producto, paso a paso, incluyendo qué pasa cuando algo sale mal (interrupción, escalación a un humano). **Gherkin NO sirve para user journeys** — Gherkin (Given/When/Then) es para **criterios de aceptación**, no para mapear el recorrido del usuario. No confundas las dos cosas.
- **North Star (Segmento 10):** entre todas tus métricas, una sola captura el valor real que entregás. Para Uber no son las descargas ni los usuarios registrados — es la **cantidad de viajes completados**. Esa es la métrica que, si sube, todo lo demás está bien. Elegí la tuya; el resto de KPIs son satélites de esa.

17.1 Paso 0 — Análisis de Conflictos (el momento clave)

Antes de empezar el Segmento 1, el prompt obliga a la AI a ejecutar un **Paso 0: Análisis de Conflictos**. Es, de lejos, **lo mejor de todo el workflow** — y es donde la mayoría de la gente tiene su primer "click".

La AI lee **todos** tus inputs (PVB, deep research, ICP, transcripciones de reuniones con clientes) y te devuelve una tabla con las **contradicciones, vacíos y supuestos incompatibles** que detectó entre tus propios documentos. Por ejemplo: en tu PVB dijiste que el usuario es el analista, pero en tu ICP el que paga es el VP de finanzas; en tu deep research aparece un competidor que tu PVB ignora; tu modelo económico asume márgenes que tu propio análisis de costos contradice.

Lo importante no es lo que el agente "resuelve" — es lo que te obliga a decidir a TI.

El agente no toma la decisión por vos. Te pone las contradicciones sobre la mesa y te fuerza a resolverlas una por una. Esas decisiones tuyas moldean todo el PRD que viene después. Si las pasás por alto acá, te van a explotar en el Segmento 9 o, peor, cuando estés construyendo.

No apures este paso. Tomate el tiempo de leer cada conflicto, pensarlo, y darle a la AI tu decisión explícita. Es el momento donde el ejercicio deja de ser "la AI me escribe un documento" y pasa a ser "la AI me ayuda a pensar".

18. Cómo co-crear tu PRD con AI

El programa tiene un **template público en GitHub** que estandariza este ejercicio. No vas a inventar tus propios prompts — vas a usar los que están en el repo y reemplazar los archivos de ejemplo con los tuyos.

18.1 El repo

```
https://github.com/hardcore-ai/c4
```

Dentro hay una carpeta `Estación 2/` con esta estructura:

```
Estación 2/
├── README.md                # Guía rápida del workflow
├── prompts-especificacion.md # Los 3 prompts secuenciales (PRD, Arquitectura,
Backlog)                    # Inputs – ejemplo trabajado (AgentVault)
├── docs/
│   ├── pvb.md
│   ├── overview.md
│   ├── mercado.md
│   ├── icp.md
│   └── critica.md
└── specs/                  # Output (vacío hasta que ejecutes el prompt)
    └── README.md
```

El ejemplo trabajado que vas a encontrar en `docs/` es **AgentVault** — un escrow autónomo en USDC para agentes (un agente deposita fondos, otro entrega el trabajo, el contrato libera el pago cuando se cumple la condición). Sirve como referencia de la **profundidad esperada** en cada input: mirá qué tan concreto está cada documento antes de escribir los tuyos.

18.2 Paso a paso

1. Clona el repo:

```
gh repo clone hardcore-ai/c4
cd c4/estacion2
```

2. Reemplaza los archivos de `docs/` con los tuyos.

Los archivos que están allí pertenecen al ejemplo trabajado (AgentVault). Sirven como referencia de la profundidad esperada. Tu trabajo es reemplazarlos con tus propios documentos:

Archivo	Tu versión
docs/pvb.md	Tu PVB refinado (sección 13 de este manual)
docs/overview.md	Tu análisis del dominio del problema
docs/mercado.md	Tu análisis de mercado, competencia, regulación
docs/icp.md	Tu Ideal Customer Profile y buyer personas
docs/critica.md	Tu deep research de crítica: huecos y riesgos

Puedes añadir tantos archivos adicionales a `docs/` como tengas: benchmarks, notas de campo, análisis técnicos. Mientras más rico el contexto, mejor el PRD.

Mete las transcripciones de tus reuniones con clientes en `docs/`. Si grabaste o transcribiste una conversación con un usuario o cliente real, esa transcripción es **oro puro** como input del PRD — captura el problema en las palabras del usuario, no en las tuyas. Pegala como un `.md` más. (Y ver el mensaje de fondo más abajo: nada reemplaza hablar con el cliente real.)

Markdown > PDF para los inputs. Convertí tus documentos a Markdown antes de meterlos en `docs/`. Opus es el modelo que vas a usar para el análisis serio, y es caro: el Markdown es mucho más eficiente en tokens que un PDF (un PDF arrastra layout, imágenes y ruido que se traduce en tokens que pagás sin que aporten). Mismo contenido, fracción del costo. Como regla: si lo podés pasar a `.md`, pasalo.

3. Abre `prompts-especificacion.md` y copia el Prompt 1 — PRD.

Es un prompt largo. Está completo, listo para pegar. NO lo modifiques.

4. Abre una conversación nueva dedicada solo a tu PRD. Pega el Prompt 1 y adjunta (o, si trabajas dentro del repo con Claude Code, deja que lea) **todos** los archivos de tu `docs/`.

Tip que te ahorra la frustración #1 de la clase — Sonnet vs Opus. Este prompt está diseñado para avanzar **segmento por segmento**, deteniéndose en cada uno a pedirte aprobación. Con **Sonnet en reasoning medium** el modelo a veces **no respeta** ese ritmo: se adelanta, te escupe varios segmentos de una, o ignora el "pará y pregúntame". Si te pasa, tenés dos salidas: 1. **Abrí una sesión nueva con Opus** (sigue el prompt segmento-por-segmento con mucha más disciplina), o 2. Decile explícitamente **"no vayamos paso a paso"** y dejá que genere el PRD de corrido, para después refinarlo.

No pelees con un modelo que no quiere ir despacio: cambiá de modelo o cambiá de modo. Es la causa #1 de frustración en esta práctica.

Si trabajás en Claude Code, tres comandos que te salvan la sesión: - `/init` — arranca el contexto del repo (lee la estructura, los `docs/`, los steering files). - `/rename` — ponle un nombre a la sesión de tu PRD para encontrarla después. - `/resume` — retomá esa misma sesión al día siguiente. El PRD se construye a lo largo de varios días; si no nombrás y retomás la sesión, perdés el hilo (y el contexto acumulado). Nombrala hoy, retomala mañana.

5. Trabaja el Paso 0 primero — sin apurarlo.

La AI te va a presentar una tabla de conflictos detectados entre tus documentos (ver sección 17.1). Resuélvelos uno por uno. **Acá no estás aprobando texto: estás tomando decisiones de producto.** El agente te muestra las contradicciones; vos decidís. Estas decisiones moldean todo el PRD que viene después. Es el paso de mayor valor de todo el ejercicio — no lo saltes.

6. Trabaja segmento por segmento.

La AI produce el Segmento 1, se detiene, te pregunta: *"¿Apruebas este segmento, tienes ajustes, o quieres que explore una dirección diferente?"*

- Si está bien → "apruébalo, sigamos con el Segmento 2"
- Si quieres ajustes → di qué cambiar
- Si la dirección está mal → corrige y pídele que rehaga

No avances hasta aprobar el actual. Esto es lo que diferencia un PRD útil de uno generado por la AI sin supervisión.

7. Cuando termines los 13 segmentos, pídele que consolide todo el documento. Guárdalo en `specs/prd.md`.

8. Generá una versión legible para humanos (Mermaid → HTML).

El `.md` es perfecto para el LLM, pero **no es cómodo de leer** para vos o tus stakeholders — y los diagramas en Mermaid aparecen como código, no como dibujos. Pedile a la AI que **genere una versión HTML** del PRD con los diagramas Mermaid renderizados. "Los markdown son buenos para el LLM, no para leer; generá la versión HTML." Si estás en Claude Code, **computer use** puede renderizar el Mermaid solo y entregarte el HTML listo para abrir en el navegador o compartir. Es la diferencia entre un entregable que nadie abre y uno que tu sponsor de verdad lee.

18.2.1 Antes del PRD: deep research de tus inputs

El PRD es tan bueno como los `docs/` que le des. Si tu deep research es flojo, el PRD va a ser flojo (basura entra, basura sale). Herramientas que Danny recomienda para armar inputs ricos antes de ejecutar el prompt:

Herramienta	Para qué brilla
Gemini (Deep Research)	La mejor a juicio de Danny para investigación profunda de mercado, competencia y dominio.
Perplexity	Respuestas con citas verificables, rápida para mapear el panorama.
Grok	Acceso a data de X (Twitter) — útil para señales en tiempo real y conversación de mercado.
NotebookLM	Cuando ya tenés tus fuentes y querés interrogarlas y sintetizarlas sin que invente nada.

El output de estas herramientas (mercado, ICP, crítica) va a `docs/` como Markdown. Mientras más rico y honesto el research, más afilado el Paso 0 — porque el agente tiene material real con qué detectar conflictos.

18.3 Para los FDE

El Segmento 3 (ICP Detallado) tiene espacio natural para tu **Stakeholder Map**: sponsor, usuario final, veto holder, aliado técnico. Cuando la AI llegue a ese segmento, asegúrate de aportar la información de tu organización específica.

Si tu situación requiere modificar el Segmento 12 (Riesgos) para incluir riesgos políticos internos (cambio de sponsor, presupuesto recortado, IT bloquea), hazlo. Es tu PRD.

18.4 Cuándo usar los Prompts 2 y 3

`prompts-especificacion.md` incluye dos prompts adicionales que **NO ejecutas esta semana**:

Prompt	Cuándo lo vas a usar	Output
Prompt 1 — PRD	Esta semana (entrega Estación 3)	specs/prd.md
Prompt 2 — Arquitectura	Estación 5 — <i>Diseñando el Cómo</i> (AI-DLC Construction, ADR)	specs/arquitectura.md
Prompt 3 — Backlog	Estaciones de implementación (E6–E7)	specs/backlog.md

Concéntrate solo en el Prompt 1 esta semana.

19. Plantilla del PRD

No hay plantilla en blanco para llenar a mano — la generas ejecutando el Prompt 1 con tus inputs. La estructura de salida son los 13 segmentos descritos en la sección 17.

Si querés ver cómo se ve un PRD generado con esta metodología, mira el ejemplo de AgentVault o de EntreVistaAI en los repos del programa.

20. Stakeholder Map (FDE)

Si eres FDE, este es tu entregable adicional. Sin él, **no vas a llegar a producción**. Recuerda: el 95% de los proyectos de GenAI en empresas no llegan a producción, y el 77% de esas fallas son organizacionales.

Tu Stakeholder Map debe identificar 4 personas con nombre y apellido:

Rol	Quién	Qué necesita	Cuándo lo voy a conversar
Sponsor	[Nombre, cargo]	[Resultado concreto]	[Antes de empezar / Semana 2 / Demo Day]
Usuario final	[Nombre, cargo]	[Beneficio operativo]	[Validación cada semana]
Veto holder	[Nombre, cargo]	[Garantía técnica / compliance]	[Conversar AHORA, no después]
Aliado técnico	[Nombre, cargo]	[Acceso a sistemas / aprobación]	[Semana 1]

Regla crítica para FDE: habla con el veto holder ANTES de tener producto. No después.
Es 10x más fácil obtener un "sí" temprano que destrabar un "no" tardío.

PARTE V — MENSAJES DE FONDO

21. Lo que de verdad te tenés que llevar

Más allá de los frameworks y del workflow, esta estación tiene dos mensajes que valen más que cualquier plantilla.

21.1 Nada reemplaza hablar con el cliente real

Podés tener el mejor deep research, el PRD más pulido y el Paso 0 más afilado. Nada de eso reemplaza **sentarte con un usuario o cliente real** y escucharlo.

"Para un producto real no hay nada que reemplace interactuar directamente con el cliente o usuario."

Por eso las transcripciones de tus reuniones con clientes son uno de los mejores inputs del PRD (sección 18.1): el problema, en las palabras de quien lo sufre, vale más que tres horas de research solo. Si todavía no hablaste con un usuario real de tu producto, esa es tu tarea más importante de la semana — por encima de cualquier documento.

21.2 Ya no más solo código

El reframe que recorre todo el programa: el rol del desarrollador se está moviendo **del código hacia el problema**.

Ya no más solo código — cada vez más cerca del problema, del negocio, del cliente, del dolor del usuario.

Esto no es retórica. Es la razón por la que existe el **FDE (Forward Deployment Engineer)**: la IA disuelve la frontera entre front, back y QA, y deja un rol transversal que va de punta a punta — del problema del negocio hasta el producto desplegado. Es un rol "*muy sexy*", full-stack end-to-end. En vez de memorizar cifras de Palantir, **andá e investigá Palantir y el rol de FDE** por tu cuenta — vas a entender por qué es uno de los roles más demandados del momento.

Anécdota — el ingeniero de Mercado Libre (C1). Un estudiante de la primera cohorte construyó, durante el programa, **AgentVault**: un escrow autónomo en USDC para agentes. *"Dos semanas después ya tenía agendada una reunión con la Vicepresidenta de Pagos de Mercado Libre para mostrarle lo que hizo, porque esa vaina va para arriba."* No fue un demo bonito: fue un producto real, sobre un problema real, que llegó a la persona correcta. Eso es lo que pasa cuando te acercás al problema en vez de quedarte solo en el código.

21.3 Dos recordatorios estratégicos

- **¿Es un prompt o es un proceso?** Antes de comprometerte con una idea, preguntate si lo que vas a construir es un *output* puntual (un prompt) o un *workflow* completo (un proceso). *"Reconciliar facturas con políticas internas no es un prompt; es todo un proceso. Eso pueda que sobreviva."* Lo que es un solo prompt, el próximo modelo te lo come. Lo que es un proceso con contexto de negocio, persiste. (Es el test de durabilidad de la Parte II, dicho en una frase.)
- **El moat es lo difícil de copiar — pensá en los chicles.** ¿Por qué Adams perdió cuota de mercado frente a Trident, si los dos son chicle? Porque Adams no tenía diferenciador. Un **moat** es justamente lo contrario: algo que hace **muy difícil que te copien o te saquen del mercado**. Sin moat, sos un chicle más.
- **Piensa en impacto, no abarques mucho.** Fue el consejo más repetido de la clase, por encima incluso de "elegí UN moat". No intentes resolver todo: elegí el problema de mayor impacto y enfocate ahí. Un producto que mueve la aguja en una cosa gana sobre uno que hace diez cosas a medias.

PARTE VI — TAREA + RECURSOS

22. Lo que debes traer al martes 1 de septiembre

Deadline: Estación 3 — martes 1 de septiembre, 7:00 PM.

Para todos

1. **PVB refinado** — todos los campos llenos o con su pregunta de investigación anotada.
2. **PRD v1** — co-creado con AI, incorporando el feedback de la práctica de la clase y, sobre todo, lo que te sacó el Análisis de Conflictos.
3. **Entorno de coding agents listo** — la Estación 3 es *Ingeniería Agéntica* con Carlos Alarcón: coding agents, MCP, skills, hooks y subagentes, hands-on. Llega con Claude Code funcionando y tus credenciales resueltas. No quemamos clase en setup.

Si además quieres ir adelantando decisiones técnicas (stack, modelo, integraciones, datos), anótalas — las vas a formalizar como ADR en la Estación 5, *Diseñando el Cómo*. No es requisito para el martes.

Adicional para FDE

4. **Stakeholder Map** — sponsor, usuario final, veto holder, aliado técnico. Diagrama simple. Lo importante es que sepas a quién tienes que convencer, en qué orden, y con qué argumento.

23. Recursos esta semana

Soporte continuo (tutorías/bonus)

C4 tiene **once espacios fijos de refuerzo**, lunes y miércoles de 7 a 8pm, uno por estación. El de la Estación 2 es el **lunes 31 de agosto**: lleva tu PRD a medio camino y lo trabajamos ahí. El **canal de la cohorte** sigue siendo tu primer recurso para cualquier duda entre sesiones.

Material de referencia

Repasa este manual (Partes II, III y IV) y los repos del programa con los ejemplos trabajados (AgentVault, EntreVistaAI) mientras refinas tu PVB y co-creas tu PRD.

ANEXOS

A. Glosario

Término	Definición
AI-DLC	AI-Driven Development Lifecycle. Metodología propuesta por AWS (julio 2025) para reemplazar el SDLC tradicional. Las 3 fases: Inception → Construction → Operations.
BMAD	Breakthrough Method for AI-Driven Development. Framework de multi-agentes con roles especializados (PM, architect, dev, QA).
Bolt	En AI-DLC, ciclo corto de horas o días. Reemplaza al sprint del SDLC.
FDE	Forward Deployment Engineer. Rol inventado por Palantir, hoy adoptado por OpenAI, Anthropic, Salesforce, Databricks. Ingeniero que entiende el negocio Y sabe construir con AI.
GSD	Get Shit Done. Framework dev-flavored para shipping con coding agents (Claude Code, Codex, Gemini CLI). Creado por TÂCHES en 2025.
Human-in-the-loop	Diseño donde la IA propone/ejecuta pero un humano valida en puntos clave (aprobaciones, escalaciones). Suele ser un <i>principio no negociable</i> del producto (Segmento 6 del PRD).
ICP / Buyer persona	Ideal Customer Profile: descripción precisa de a quién le sirve tu producto (firmographics: industria, tamaño, geografía). La <i>buyer persona</i> es el retrato del individuo que decide o usa dentro de ese perfil (rol, dolores, triggers, objeciones). Cuanto más específico, mejor el PRD.
Job to be Done (JTBD)	El "trabajo" que el usuario contrata a tu producto para resolver. La gente no compra un taladro, compra el hueco — en realidad, el cuadro colgado. Define el producto por el trabajo, no por sus features.
MADR / ADR	Markdown Architectural Decision Records. Formato para documentar decisiones de arquitectura en Markdown. En C4 se trabaja en la Estación 5, <i>Diseñando el Cómo</i> .
MoSCoW	Método de priorización del MVP: Must / Should / Could / Won't . Define qué entra a v1 (Must) y qué queda en el roadmap, sin matarlo.

Término	Definición
MVP	Minimum Viable Product. La versión más pequeña del producto que demuestra el valor central. Se prioriza con MoSCoW.
Mob session	En AI-DLC, sesión donde todo el equipo + la IA validan en tiempo real. Reemplaza al trabajo aislado.
Moat	Ventaja competitiva que se fortalece con el tiempo y que hace muy difícil que te copien o te saquen del mercado (no se replica con dinero ni con la misma API). Los tres que importan: Data, Distribution, Trust. Analogía: Adams perdió frente a Trident por no tener moat.
North Star	La única métrica que captura el valor real que entrega tu producto. Si sube, todo lo demás está bien. Ejemplo: para Uber, la cantidad de viajes completados.
PMF	Product-Market Fit. Cuando hay evidencia clara de que el producto resuelve un problema real para un mercado que está dispuesto a pagar.
PRD	Product Requirements Document. Documento que describe el producto completo (problema, ICP, diferenciación, casos de uso, MVP, métricas) y sirve de contexto para co-construir con AI.
PVB	Product Vision Board. Canvas de una página que captura la estrategia del producto antes de construir.
Red Ocean	Mercado existente saturado de competidores. Lo opuesto de Blue Ocean (mercado nuevo).
Spec-Driven Development	Approach donde el PRD/spec detallado es el contrato con el código.
UoW (Unit of Work)	En AI-DLC, paquete pequeño y verificable. Reemplaza al epic.
User journey	El recorrido paso a paso de un usuario real por tu producto, incluyendo qué pasa cuando algo sale mal (interrupción, escalación a humano). Gherkin (Given/When/Then) NO sirve para esto — Gherkin es para criterios de aceptación.
Vibe Coding	Approach de prompt directo, sin spec formal, iteración rápida. Bueno para spikes.
Wrapper	Producto que es solo una capa de UX encima de la API de otro (ej. una UI bonita sobre GPT). Sin moat real: cualquiera lo replica en un fin de semana. Fue la causa de muerte de Jasper.

B. Fuentes y lecturas adicionales

Sobre los casos de estudio

- **Chegg:** [European Business Magazine — Chegg Lost \\$14B to ChatGPT](#)
- **Jasper:** [Jasper AI Statistics 2026](#)
- **Duolingo:** [Fortune — Duolingo CEO admits memo controversy](#)
- **Perplexity:** [TechCrunch — Perplexity raised \\$200M at \\$20B valuation](#)
- **Clay:** [Clay Series C announcement](#)

Sobre el Forward Deployment Engineer

- [Gigged.ai — Forward Deployed Engineer: 2026's Hottest Job Title](#)
- [The Pragmatic Engineer — What are Forward Deployed Engineers](#)
- [MIT Sloan — Why 95% of Corporate AI Projects Fail](#)

Sobre AI-DLC

- [AWS DevOps Blog — AI-Driven Development Life Cycle \(Raja SP, jul 2025\)](#)
- [AWS DevOps Blog — Open-Sourcing Adaptive Workflows for AI-DLC](#)
- [GitHub — awslabs/aidlc-workflows](#)

Sobre los frameworks

- [GSD official site](#)
 - [Pulumi — Picking the Right Framework \(Superpowers/GSD/GSTACK\)](#)
 - [BMAD Method — GitHub](#)
-

C. Plantilla del PVB (en blanco)

PRODUCT VISION BOARD – [Nombre del producto]

PROBLEMA (que persiste post-GPT-6)

Durability Score (1-5): ____

SEGMENTO TARGET

VETO HOLDER (quién puede matar la adopción aunque el end-user ame el producto)

MOAT PRIMARIO

☐ Data Moat ☐ Distribution Moat ☐ Trust Moat

¿Qué [data/distribución/trust] única poseemos o podemos construir?

ARENA COMPETITIVA

☐ Pioneer (AI-Native)
☐ Disruptor (AI-Disrupted)
☐ Enhancer (AI-Enhanced)

UX PARADIGM

☐ Assistant
☐ Agent
☐ Autonomous
☐ Embedded

MODELO ECONÓMICO

Pricing model: _____

Costo por usuario / mes: \$_____

Revenue por usuario / mes: \$_____

Gross margin proyectado: _____%

(FDE) ROI esperado: _____

3 RIESGOS CRÍTICOS

1. _____
2. _____
3. _____

D. PRD — no hay plantilla en blanco

A diferencia del PVB, el PRD **no se llena a mano**. Se **genera ejecutando el Prompt 1** de `prompts-especificacion.md` del repo `hardcore-ai/c4`, alimentado con tus inputs en la carpeta `docs/`.

Ver sección 17 de este manual para los 13 segmentos que la AI va a producir, y sección 18 para el paso a paso operativo del workflow.

Si quieres ver cómo se ve un PRD completo generado con esta metodología, mira el ejemplo de **EntreVistaAI** (`hardcore-ai/EntreVistaAI` → `specs/prd.md`).

E. Mapas mentales del programa (Referencia — no se dicta en vivo)

Esto NO se dicta en la sesión. Es material de referencia para cuando, más adelante, te preguntes "¿cómo llevo mi PRD a código sin perder contexto?" y "¿dónde encaja todo esto en el programa?". El detalle de los frameworks de código entra en las **estaciones de Implementación (E4-E6)**. Hoy no eliges framework: hoy solo sabes que el mapa existe.

E.1 Landscape de frameworks (cuándo entra cada uno)

Cuando termines tu PRD, hay un ecosistema de frameworks (2025-2026) para llevar la especificación a código. No tienes que elegir hoy:

Framework	En una línea	Cuándo
Spec-Driven Development	PRD → spec detallado → código, todo trazable	Producción crítica, regulación (healthcare, fintech, legal)
BMAD (Breakthrough Method for AI-Driven Development)	Múltiples agentes con roles (PM, architect, dev, QA)	Proyectos complejos con muchos stakeholders; necesitas varias perspectivas sin tener el equipo
GSD (Get Shit Done)	Slash commands + sub-agentes frescos + estado en Markdown; dev-flavored, lean (~60K stars)	Shipping con Claude Code / Codex / Gemini CLI. Natural fit del programa
Vibe Coding	Prompt directo, sin spec, iteración rápida	Spike de 1-2 días para validar una idea antes de comprometerte

E.2 AI-DLC: la metodología del programa

El programa sigue **AI-DLC (AI-Driven Development Lifecycle)**, la metodología que AWS propuso (julio 2025) para reemplazar el SDLC tradicional. Principio: **los humanos deciden y validan; la IA planifica y ejecuta.**

- **3 fases:** Inception → Construction → Operations.
 - **Micro-loop (en cada fase):** Human Task → AI Plan → Human Approves → AI Executes → Human Verifies → ∞.
 - **Vocabulario vs SDLC:** Bolts (en vez de sprints), Units of Work / UoW (en vez de epics), IA como teammate (no herramienta auxiliar), Mob sessions (equipo + IA en tiempo real).
 - **Cómo encaja en el journey C4:** Inception = E1–E5 (producto → el QUÉ → el CÓMO) · Construction = E6–E7 (del spec al harness, del backlog al merge) · Operations = E8–E11 (calidad, despliegue, seguridad, observabilidad) · E12 = Demo Day. **El PVB y el PRD de esta semana son Inception.**
-

AI for Developers | 30X · Cohorte 4 · Estación 2 · Agosto 2026